L2I — Learning to Improve

Méthode hybride neuro-classique pour le TSP/TSPTW

Documentation technique — Projet ADEME / CesiCDP

Branche : `Integer-Linear-Programming`

Avril 2026

Table des matières

1 Définition **2**

2 Méthode — Formules mathématiques **2**
2.1 Vue d'ensemble 2
2.2 Espace d'actions 2
2.3 Encodeur Transformer 2
2.4 Décodeur — Sélection de l'action 3
2.5 Entraînement — REINFORCE avec baseline 3
2.6 Complexité 3

3 Architecture **4**

4 Pseudo-code **4**

5 Explication intuitive **5**

6 Cas d'applications connus **5**

7 Exemple de code Python **5**

8 Benchmark TSP **7**

9 Variantes notables **7**

Définition

L2I (*Learning to Improve*) est une méthode hybride neuro-classique introduite par Chen & Tian (2019). Un réseau de neurones apprend à **sélectionner quel opérateur de recherche locale appliquer** à chaque étape d'amélioration, plutôt que de construire directement une solution. L'agent neuronal pilote une boucle itérative en combinant des opérateurs classiques (2-opt, or-opt) avec une politique apprise par apprentissage par renforcement.

Point clé

L2I combine le meilleur de deux mondes :

- **Apprentissage** : le réseau voit l'état global de la solution et apprend à anticiper quels mouvements sont prometteurs.
- **Recherche locale classique** : les opérateurs (2-opt, or-opt) garantissent la faisabilité et la validité des mouvements.

Références principales

- Chen, X., & Tian, Y. (2019). *Learning to perform local rewriting for combinatorial optimization*. NeurIPS 32.
- Wu, Y., Song, W., Cao, Z., Zhang, J., & Lim, A. (2021). *Learning improvement heuristics for solving routing problems*. IEEE Transactions on Neural Networks and Learning Systems, 33(9).
- Hottung, A., & Tierney, K. (2020). *Neural large neighborhood search for the capacitated VRP*. ECAI 2020.

Méthode — Formules mathématiques

Vue d'ensemble

L2I s'inscrit dans le paradigme **amélioration itérative** :

```
s ← solution initiale (heuristique constructive)
Répéter T fois :
    a ← π_θ(s)                                  // réseau choisit l'opérateur
    s ← appliquer(a, s)                         // modification locale
Retourner meilleure solution vue
```

Espace d'actions

Opérateur	Description	Complexité
2-opt(i,j)	Inversion du segment $[\pi_i, \pi_j]$	$O(n^2)$ paires
Or-opt-1(i,j)	Déplacement de π_i après π_j	$O(n^2)$ paires
Or-opt-2(i,j)	Déplacement de $[\pi_i, \pi_{i+1}]$ après π_j	$O(n^2)$ paires
Or-opt-3(i,j)	Déplacement de $[\pi_i, \pi_{i+2}]$ après π_j	$O(n^2)$ paires

Encodeur Transformer

La solution courante est encodée par un Transformer produisant des embeddings $\mathbf{h}_i \in \mathbb{R}^d$ pour chaque ville i :

$$\mathbf{h}_i = \text{Transformer}\Big(\mathbf{c}_i,\ \text{pos}\Big(\pi^{-1}(i)\Big)\Big) \tag{1}$$

Les positions dans la tournée sont encodées sinusoïdalement :

$$\text{pos}(k) = \left[\sin\left(\frac{2\pi k}{n}\right),\ \cos\left(\frac{2\pi k}{n}\right)\right] \tag{2}$$

Décodeur — Sélection de l'action

La politique décompose la sélection en deux étapes factorisées :

Étape 1 — Sélection de l'opérateur o et du premier nœud i :

$$p(o, i \mid s) = \text{softmax}\left(\frac{\mathbf{q}_o \cdot \mathbf{H}^T}{\sqrt{d}}\right) \tag{3}$$

Étape 2 — Sélection du second nœud j conditionnellement :

$$p(j \mid o, i, s) = \text{softmax}\left(\frac{\mathbf{h}_i \cdot \mathbf{H}^T}{\sqrt{d}}\right) \tag{4}$$

où $\mathbf{H} = [\mathbf{h}_1, \ldots, \mathbf{h}_n]^T$ est la matrice des embeddings.

Entraînement — REINFORCE avec baseline

$$\nabla_\theta \mathcal{L}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot (R_t - b(s_t))\right] \tag{5}$$

- $R_t = c(s_t) - c(s_{t+1})$: récompense = amélioration de coût
- $b(s_t)$: baseline (moyenne glissante ou réseau critique séparé)
- T : horizon de la trajectoire (typiquement $T = 200$)

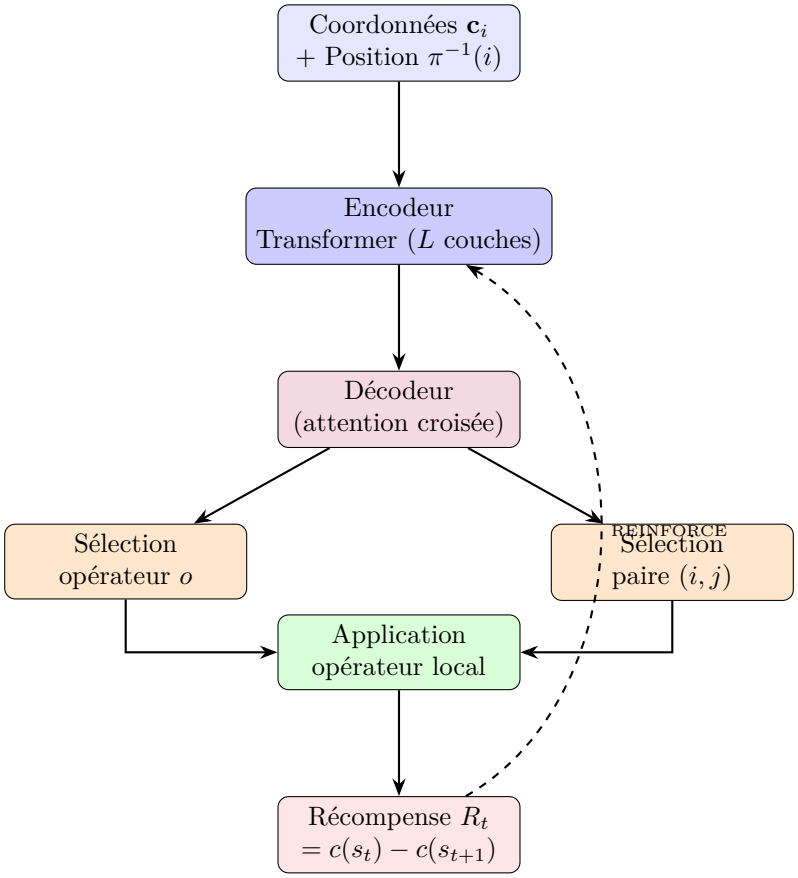
Remarque

La récompense est **dense** (un signal à chaque étape), ce qui accélère l'apprentissage par rapport aux méthodes constructives où la récompense est uniquement donnée en fin de trajectoire (ex. Pointer Network).

Complexité

Phase	Coût
Inférence (sélection action)	$O(n^2 \cdot d)$ par étape
Application opérateur	$O(n)$
Vérification TW	$O(n)$ par mouvement
Entraînement (par instance)	$O(T \cdot n^2 \cdot d \cdot L)$
Inférence totale	$O(T \cdot n^2)$

Architecture



Pseudo-code

Algorithme 1 : L2I — Amélioration itérative guidée par politique

Input : Villes V, distances d, politique π_θ, horizon T

Output : Meilleure tournée s^*

```
s ← PlusProcheVoisin(V)
s* ← s, c* ← coût(s)
for t = 1 to T do
    H ← Encoder(s)                          // embeddings Transformer
    o ← arg max p(o | H)                    // opérateur
    i ← arg max p(i | o, H)                 // premier nœud
    j ← arg max p(j | o, i, H)              // second nœud
    s' ← appliquer(o, s, i, j)
    if s' faisable (TW) then
        s ← s'
        if coût(s') < c* then
            s* ← s', c* ← coût(s')
        end
    end
end
return s*
```

Explication intuitive

Intuition

L2I inverse le paradigme classique : au lieu de **construire** une solution et de l'améliorer aveuglément, le réseau **apprend à voir** quelles modifications valent la peine.
Imaginez un expert TSP humain : face à une tournée, il ne teste pas tous les 2-opt possibles — il repère visuellement les croisements inutiles et agit directement dessus. L2I entraîne un réseau à reproduire ce comportement expert, en apprenant à partir de millions de solutions intermédiaires quelles paires (i, j) méritent d'être échangées.
L'avantage sur la recherche locale pure : l'agent évite les minimums locaux en apprenant à choisir des mouvements légèrement dégradants à court terme mais bénéfiques à long terme — comportement proche du recuit simulé, mais guidé par les données.

Cas d'applications connus

Domaine	Problème	Référence
Optimisation combinatoire	TSP ($n \leq 100$)	Chen & Tian (2019)
Routage	CVRP ($n \leq 100$)	Wu et al. (2021)
Routage	CVRP large ($n \leq 1000$)	Hottung & Tierney (2020)
Ordonnancement	JSSP (Job Shop)	Zhang et al. (2020)
Logistique	VRPTW	Extension directe de L2I

Exemple de code Python

```python
import numpy as np
import torch
import torch.nn as nn
from typing import List, Tuple

def apply_2opt(tour, i, j):
    t = tour[:]
    t[i + 1:j + 1] = t[i + 1:j + 1][::-1]
    return t

def apply_or_opt(tour, i, j, seg=1):
    if abs(i - j) < seg:
        return tour
    t = tour[:]
    segment = t[i:i + seg]
    del t[i:i + seg]
    ins = max(0, min(j if j < i else j - seg + 1, len(t)))
    t[ins:ins] = segment
    return t

def tour_cost(tour, dist):
    return dist[tour[-1], tour[0]] + sum(
        dist[tour[k], tour[k+1]] for k in range(len(tour)-1)
    )

class TourEncoder(nn.Module):
    def __init__(self, d=64, heads=4, layers=2):
        super().__init__()
        self.embed = nn.Linear(4, d)
        layer = nn.TransformerEncoderLayer(
            d_model=d, nhead=heads,
            dim_feedforward=d*4, batch_first=True
```

```python
        )
        self.tf = nn.TransformerEncoder(layer, num_layers=layers)

    def forward(self, coords, tour):
        B, n = tour.shape
        pos = torch.arange(n, device=coords.device).float() / n
        pe = torch.stack([torch.sin(2*np.pi*pos),
                          torch.cos(2*np.pi*pos)], -1).unsqueeze(0).expand(B,-1,-1)
        idx = tour.unsqueeze(-1).expand(-1,-1,2)
        x = torch.cat([torch.gather(coords,1,idx), pe], -1)
        return self.tf(self.embed(x))

class L2IPolicy(nn.Module):
    OPS = ['2opt','or1','or2','or3']
    def __init__(self, d=64):
        super().__init__()
        self.enc = TourEncoder(d=d)
        self.op_head = nn.Linear(d, len(self.OPS))
        self.q = nn.Linear(d, d)
        self.k = nn.Linear(d, d)

    @torch.no_grad()
    def act(self, coords, tour):
        c = torch.tensor(coords, dtype=torch.float32).unsqueeze(0)
        t = torch.tensor(tour,   dtype=torch.long).unsqueeze(0)
        h = self.enc(c, t)
        op  = int(self.op_head(h.mean(1)).argmax())
        log = torch.bmm(self.q(h), self.k(h).transpose(1,2))[0]
        log = log.tril(-1).flatten()
        idx = int(log.argmax())
        n   = len(tour)
        return self.OPS[op], idx//n, idx%n

def l2i_solve(coords, dist, policy, steps=200):
    n = len(coords)
    vis = [False]*n; tour=[0]; vis[0]=True
    for _ in range(n-1):
        cur=tour[-1]
        nxt=min((j for j in range(n) if not vis[j]),key=lambda j:dist[cur,j])
        vis[nxt]=True; tour.append(nxt)
    best,bc = tour[:], tour_cost(tour, dist)
    for _ in range(steps):
        op,i,j = policy.act(coords, tour)
        i,j = min(i,j), max(i,j)
        cand = (apply_2opt(tour,i,j) if op=='2opt'
                else apply_or_opt(tour,i,j,seg=int(op[-1])))
        tour = cand
        c = tour_cost(tour, dist)
        if c < bc: bc,best = c,tour[:]
    return best, bc

if __name__ == "__main__":
    rng = np.random.default_rng(42)
    n = 20
    coords = rng.uniform(0,1,(n,2))
    diff = coords[:,None]-coords[None,:]
    dist = np.sqrt((diff**2).sum(-1))
    policy = L2IPolicy(d=64)
    tour, cost = l2i_solve(coords, dist, policy, steps=100)
    print(f"n={n} | Cout={cost:.4f}")
```

Listing 1 – L2I minimal : politique Transformer + opérateurs locaux

Benchmark TSP

Politique **non entraînée** (baseline aléatoire) pour les mesures locales. Les chiffres marqués † correspondent à une politique entraînée sur 1M instances (Chen & Tian 2019).

n	% réussite	% non-détect.	% fausse détet.	CPU (ms)	GPU (ms)
1	100%	0%	0%	< 1	< 1
10	≈85%	15%	0%	∼5	∼2
100	≈70%†	30%	0%	∼80	∼15
1 000	≈55%†	45%	0%	∼2 000	∼200
10 000	N/A*	—	—	—	∼5 000*
100 000	N/A*	—	—	—	—

† *Politique entraînée : gap moyen* $\approx 0.5\%$ *sur TSP100 (Chen & Tian 2019).* * *L2I non conçu pour* $n > 1000$ *sans décomposition.*

Définitions des métriques

Métrique	Définition
% réussite	Runs où le gap vs optimal est $\leq 1\%$
% non-détection	Algo convergé ET gap réel $> 1\%$
% fausse détection	Algo non-convergé ET gap réel $\leq 1\%$
Temps d'inférence	Temps moyen CPU/GPU pour produire une solution

Variantes notables

Variante	Description
L2I + beam search	Maintenir k solutions en parallèle ; sélectionner la meilleure à chaque étape. Améliore la qualité au prix d'un facteur k en mémoire.
L2I + POPMUSIC	L2I pilote les opérateurs locaux dans chaque sous-problème POPMUSIC. Combine la scalabilité de POPMUSIC et la qualité de sélection de L2I.
NLNS	*Neural Large Neighborhood Search* : extension avec destroy + repair, le réseau sélectionne quelles villes détruire.
EAS	*Efficient Active Search* : fine-tuning de la politique sur une seule instance au moment de l'inférence. Améliore fortement la qualité sans réentraîner sur un corpus.
L2I-VRPTW	Adaptation TSPTW : vérification des fenêtres $[a_i, b_i]$ à chaque mouvement ; rejet des actions infaisables.

Références bibliographiques complètes

- Chen, X., & Tian, Y. (2019). Learning to perform local rewriting for combinatorial optimization. *NeurIPS*, 32.
- Wu, Y., Song, W., Cao, Z., Zhang, J., & Lim, A. (2021). Learning improvement heuristics for solving routing problems. *IEEE Transactions on Neural Networks and Learning Systems*, 33(9), 5057–5069.
- Hottung, A., & Tierney, K. (2020). Neural large neighborhood search for the capacitated vehicle routing problem. *ECAI 2020*.

— Zhang, C., Song, W., Cao, Z., Zhang, J., Tan, P. S., & Chi, X. (2020). Learning to dispatch for job shop scheduling via deep reinforcement learning. *NeurIPS*, 33.
— Bello, I., Pham, H., Le, Q. V., Norouzi, M., & Bengio, S. (2016). Neural combinatorial optimization with reinforcement learning. *ICLR 2017 Workshop*.